

IA

SERVICIOS COGNITIVOS EN LA NUBE

RECONOCIMIENTO DE
FORMULARIOS

1.	EL SERVICIO DE RECONOCIMIENTO DE DOCUMENTOS.....	1
2.	MODELOS Y TAREAS SOPORTADAS POR EL SERVICIO	1
3.	IDIOMAS SOPORTADOS	3
4.	FACTURACIÓN	3
5.	FORM RECOGNIZER STUDIO.....	3
6.	UTILIZACIÓN DEL SDK PARA C# DE FORM RECOGNIZER	6
6.1.	USO DEL MODELO "GENERAL DOCUMENT"	12
6.2.	OTROS MODELOS.....	15

1. EL SERVICIO DE RECONOCIMIENTO DE DOCUMENTOS

Azure expone un servicio en la nube de **reconocimiento de formularios y documentos**, **Form Recognizer**, que permite extraer la información que contienen los documentos sometidos a análisis.

El sistema es capaz de diferenciar el contenido del documento de su estructura y puede extraer **contenido textual**, en formato de **pares clave-valor**, **entradas de selección** (marcado/no marcado) y datos en formato de **tabla**.

El algoritmo puede generar una salida en formato JSON, por lo que puede ser integrada con facilidad en cualquier aplicación u origen de datos.

Asimismo, el servicio también dispone de una funcionalidad de reconocimiento de tipos de documento. Esta funcionalidad puede ser entrenada con un conjunto de documentos de prueba para que el sistema pueda identificar un documento arbitrario.

El servicio proporciona modelos de reconocimiento preentrenados, así como la posibilidad de entrenar el reconocimiento de documentos específicos.

2. MODELOS Y TAREAS SOPORTADAS POR EL SERVICIO

El servicio da soporte a los siguientes modelos de IA:

- **Modelo de documento general** (*General document*): analiza y extrae texto, tablas, estructuras, pares clave-valor y entidades de un documento.
- **Modelo de diseño** (*Layout*): analiza y extrae tablas, líneas, palabras y marcas de selección, como botones de radio y casillas en documentos de formularios, sin necesidad de entrenar un modelo.
- **Modelo precompilado**: analiza y extrae campos comunes de tipos de documentos específicos mediante un modelo entrenado previamente.

En función de la tarea que queremos realizar, deberemos seleccionar uno u otro modelo, de acuerdo a los siguientes criterios:

- Reconocimiento y extracción de información de documentos como facturas, recibos, albaranes y tarjetas de visita.
 - Si el documento está en inglés, debe utilizarse la característica "Invoice, Receipt or Business Card".
 - Si el documento está en otro idioma, debe utilizarse la característica "Layout" o "General document".

- Documento de identificación.
 - Si el documento es un permiso de conducción estadounidense, o bien un pasaporte internacional, debe utilizarse la característica ID document
 - En caso contrario, debe utilizarse la característica "Layout" o "General document".
- Formulario o documento general
 - Si el formulario está en un formato estándar utilizado comúnmente en industria o negocios (como, por ejemplo, un Incoterm), debe usarse la característica "Layout" o "General document".
 - En caso contrario, será necesario entrenar y generar un modelo específicamente adaptado al documento.

Estos son los modelos y características disponibles a febrero de 2022:

Form Recognizer model data extraction

Model	Text extraction	Key-Value pairs	Selection Marks	Tables	Entities
 Read	✓				
 General document	✓	✓	✓	✓	✓
Layout	✓		✓	✓	
Invoice	✓	✓	✓	✓	
Receipt	✓	✓			
ID document	✓	✓			
Business card	✓	✓			
Custom	✓	✓	✓	✓	✓

3. IDIOMAS SOPORTADOS

Puedes encontrar un listado de los idiomas soportados en cada característica en el siguiente enlace: <https://docs.microsoft.com/en-us/azure/applied-ai-services/form-recognizer/language-support>

4. FACTURACIÓN

La información de facturación se encuentra disponible en el siguiente enlace: <https://azure.microsoft.com/es-es/pricing/details/form-recognizer/>. La facturación se realiza en función del número de páginas analizadas / mes.

En función del tipo de servicio contratado se aplican restricciones a la carga de trabajo. Estas restricciones se encuentran detalladas en este enlace: <https://docs.microsoft.com/en-us/azure/applied-ai-services/form-recognizer/service-limits>

5. FORM RECOGNIZER STUDIO

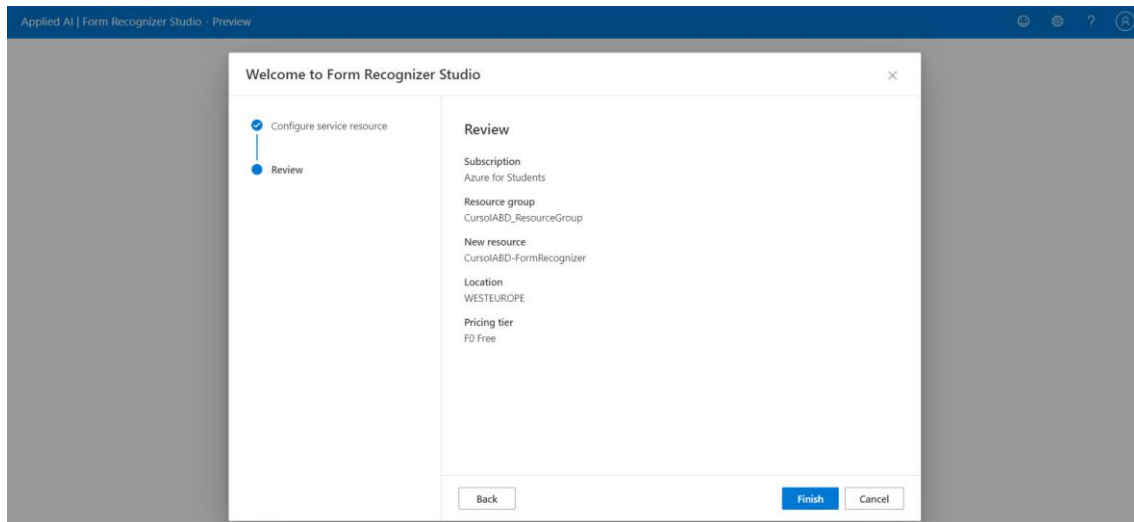
Se trata de una herramienta online que expone la funcionalidad de reconocimiento de documentos a través de una interfaz de usuario. Resulta adecuada para un uso puntual, cuando se trabaja con documentos sueltos o con baja carga de trabajo.

Antes de poder utilizar esta herramienta por primera vez, será necesario configurar el servicio y las opciones de facturación, como se muestra en las siguientes pantallas:

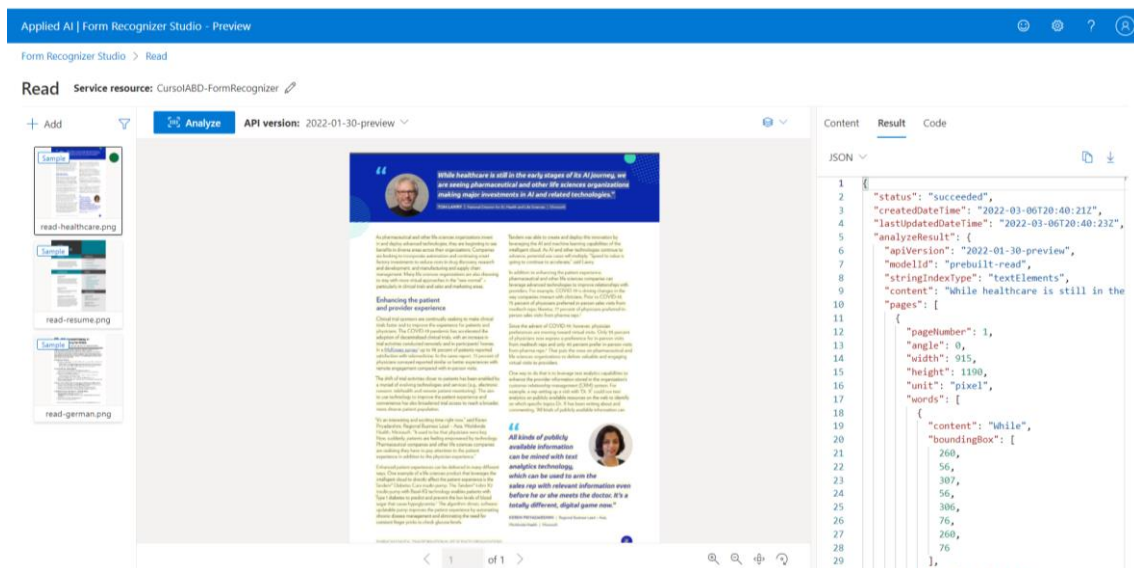
The screenshot shows the 'Welcome to Form Recognizer Studio' configuration window. The window has a title bar 'Applied AI | Form Recognizer Studio - Preview'. Inside, there's a progress bar on the left with two steps: 'Configure service resource' (selected) and 'Review'. The main area is titled 'Configure service resource' and contains the following fields and options:

- Subscription ***: A dropdown menu showing 'Azure for Students'.
- Resource group ***: A dropdown menu showing '(New) CursorABD_ResourceGroup'.
- Form Recognizer or Cognitive Service Resource ***: A dropdown menu showing 'No available options'.
- Create new resource**: A checkbox that is checked.
- Name ***: A text input field containing 'CursorABD-FormRecognizer'.
- Location ***: A dropdown menu showing 'WESTEUROPE'.
- Pricing tier ***: A dropdown menu showing 'F0 Free'.

At the bottom of the form, there are two buttons: 'Continue' (in blue) and 'Cancel' (in grey).



A continuación se muestra un ejemplo de la funcionalidad de lectura (extracción de texto). Tras seleccionar la imagen o el documento a analizar, el servicio devuelve (panel de la derecha) el contenido de texto en formato de líneas, así como un esquema de la estructura del documento en la que se recogen las *bounding boxes* o áreas de reconocimiento (rectángulos imaginarios que delimitan cada área de texto identificada), incluyendo sus coordenadas y contenido.



La siguiente pantalla muestra la funcionalidad de Layout (esquema del documento), que permite identificar la estructura del documento y extraer contenido de tablas, zonas resaltadas y texto manuscrito:

The screenshot shows the 'Layout' view in Form Recognizer Studio. The main document is a checklist titled 'Share with'. It lists various document types and the person to share them with. A 'Table' popup is displayed, showing the extracted data from the document.

Document	Share with
Adoption or guardianship papers* ☐	Personal attorney, executor
Annuity contracts ☒	Financial advisor
Bank account documents ☒	Add name(s) Ardak Sultan
Bank loan agreements ☐	Add name(s)
Bank statements ☒	Add name(s) Parvesh Giri
Birth certificates* ☐	Add name(s)
Business licence* ☒	Add name(s) Jerome Rivard

Probablemente, el modelo más interesante es el de reconocimiento de documento general, que se ofrece a través de la siguiente interfaz:

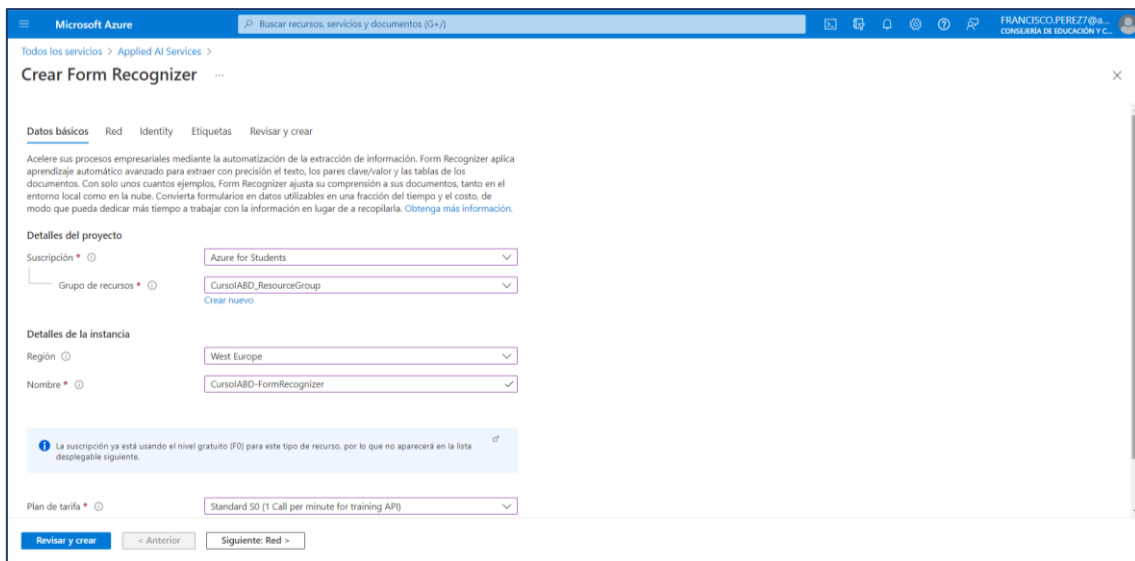
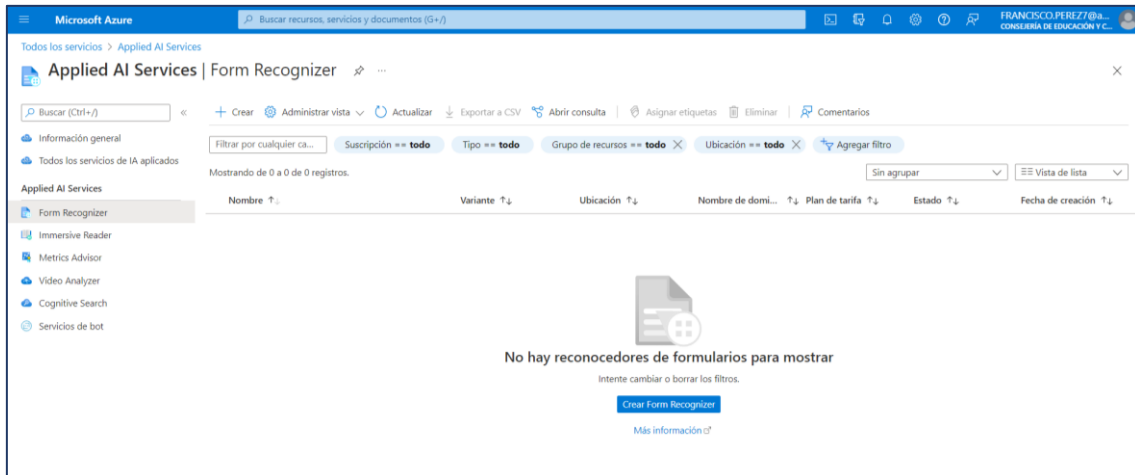
The screenshot shows the 'General Document' view in Form Recognizer Studio. The main document is a form titled 'Application for Permit to Drill (APD)'. The form has multiple sections with various fields. A 'Key-Value pairs' table is visible on the right, showing extracted data from the form.

Key	Value	Confidence
3. OPERATOR NAME and ADDRESS	Pipelines Inc. Main Street Seattle WA 98011	91.90%
NAME and ADDRESS	Main Street Seattle WA 98011	85.70%

En este caso, además de la salida en formato JSON, que incluye los bounding boxes, también tenemos una salida formato de pares [clave, valor] con cada uno de los campos del formulario.

6. UTILIZACIÓN DEL SDK PARA C# DE FORM RECOGNIZER

Antes de poder interactuar con los servicios en la nube, es necesario crear un recurso en Azure. En el siguiente enlace se indica el modo de crear el recurso, tanto en modo comando como a través del sitio web de Azure: <https://docs.microsoft.com/en-us/dotnet/api/overview/azure/ai.formrecognizer-readme?view=azure-dotnet#create-a-cognitive-services-or-form-recognizer-resource>.



Microsoft Azure
Buscar recursos, servicios y documentos (0+3)
FRANCISCO PEREZ77@...
CONSULERA DE EDUCACIÓN Y C...

Todos los servicios > Applied AI Services >

Crear Form Recognizer

Datos básicos
Red
Identity
Etiquetas
Revisar y crear

Configure la seguridad de red para su recurso de Cognitive Services.

Tipo *

- Todas las redes, incluso Internet, pueden acceder a este recurso.
- Hay redes seleccionadas. Configure la seguridad de red para su recurso de Cognitive Services.
- Se ha deshabilitado, por lo que ninguna red puede acceder a este recurso. La única manera de hacerlo es, una vez creado el recurso, configurar conexiones de puntos de conexión privados.

Revisar y crear
< Anterior
Siguiente: Identity >

Microsoft Azure
Buscar recursos, servicios y documentos (0+3)
FRANCISCO PEREZ77@...
CONSULERA DE EDUCACIÓN Y C...

Todos los servicios > Applied AI Services >

Crear Form Recognizer

Datos básicos
Red
Identity
Etiquetas
Revisar y crear

Identidad administrada asignada por el sistema

Estado

- Desactivado
- Activado

Identidad administrada asignada por el usuario

+ Agregar
Quitar

Nombre	↑↓ grupo de recursos	↑↓ suscripción	↑↓
No hay identidades administradas asignadas por el usuario asignadas a este recurso. Seleccione "Agregar" para agre...			

Revisar y crear
< Anterior
Siguiente: Etiquetas >

Microsoft Azure
Buscar recursos, servicios y documentos (0+3)
FRANCISCO PEREZ77@...
CONSULERA DE EDUCACIÓN Y C...

Todos los servicios > Applied AI Services >

Crear Form Recognizer

Validación superada

TERMS

By clicking "Crear", I (a) agree to the legal terms and privacy statement(s) associated with the Marketplace offering(s) listed above; (b) authorize Microsoft to bill my current payment method for the fees associated with the offering(s), with the same billing frequency as my Azure subscription; and (c) agree that Microsoft may share my contact, usage and transactional information with the provider(s) of the offering(s) for support, billing and other transactional activities. Microsoft does not provide rights for third-party offerings. See the [Azure Marketplace Terms](#) for additional details.

Datos básicos

Suscripción	Azure for Students
Región	West Europe
Nombre	CursoABD-FormRecognizer
Plan de tarifa	Standard S0 (1 Call per minute for training API)

Red

Tipo	Todas las redes, incluso Internet, pueden acceder a este recurso.
------	---

Identity

Tipo de identidad	None
-------------------	------

Crear
< Anterior
Siguiente
Descargar una plantilla para la automatización

Microsoft Azure

Buscar recursos, servicios y documentos (3+)

Todos los servicios >

Microsoft.CognitiveServicesFormRecognizer-20220306225330 | Información general

Implementación

Eliminar Cancelar Volver a implementar Actualizar

Buscar (Ctrl+F)

Información general

Entradas

Salidas

Plantilla

Nos encantaría recibir sus comentarios. →

Se completó la implementación

Nombre de implementación: Microsoft.CognitiveServicesFormReco... Hora de inicio: 6/3/2022, 22:57:11
Suscripción: Azure for Students Id. de correlación: 051dcb07-2164-4ec7-9e5f-539ca26c9dc
Grupo de recursos: CursoIABD_ResourceGroup

Detalles de implementación (Descargar)

Recurso	Tipo	Estado	Detalles de la operación
CursoIABD-FormRecognizer	Microsoft.CognitiveServices/accounts	OK	Detalles de la operación

Pasos siguientes

[Ir al recurso](#)

Microsoft Azure

Buscar recursos, servicios y documentos (3+)

Todos los servicios > Microsoft.CognitiveServicesFormRecognizer-20220306225330

Microsoft.CognitiveServicesFormRecognizer-20220306225330 | Entradas

Implementación

Buscar (Ctrl+F)

Información general

Entradas

Salidas

Plantilla

name

CursoIABD-FormRecognizer

location

westeurope

resourceGroupName

CursoIABD_ResourceGroup

resourceGroupId

/subscriptions/bba589b2-277e-4c82-976f-d42f6969c232/resourceGroups/CursoIABD_ResourceGroup

sku

S0

tagValues

{}

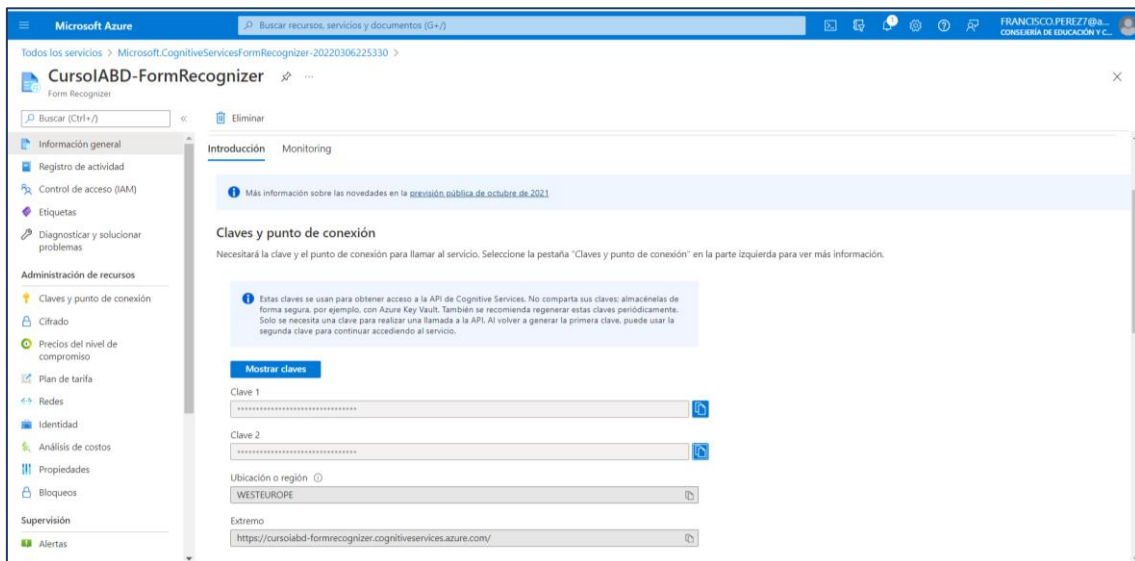
virtualNetworkType

None

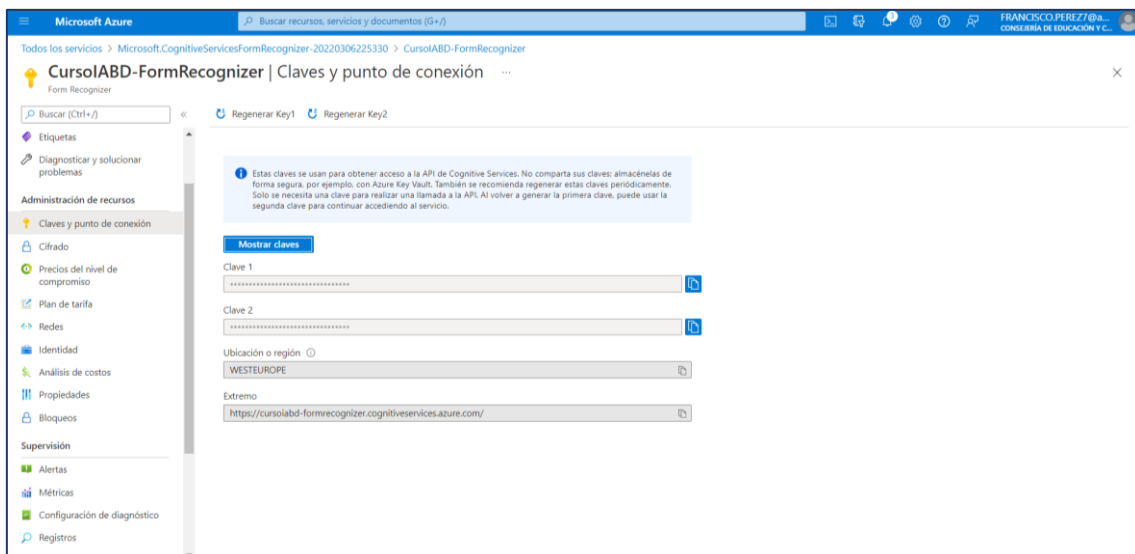
vnet

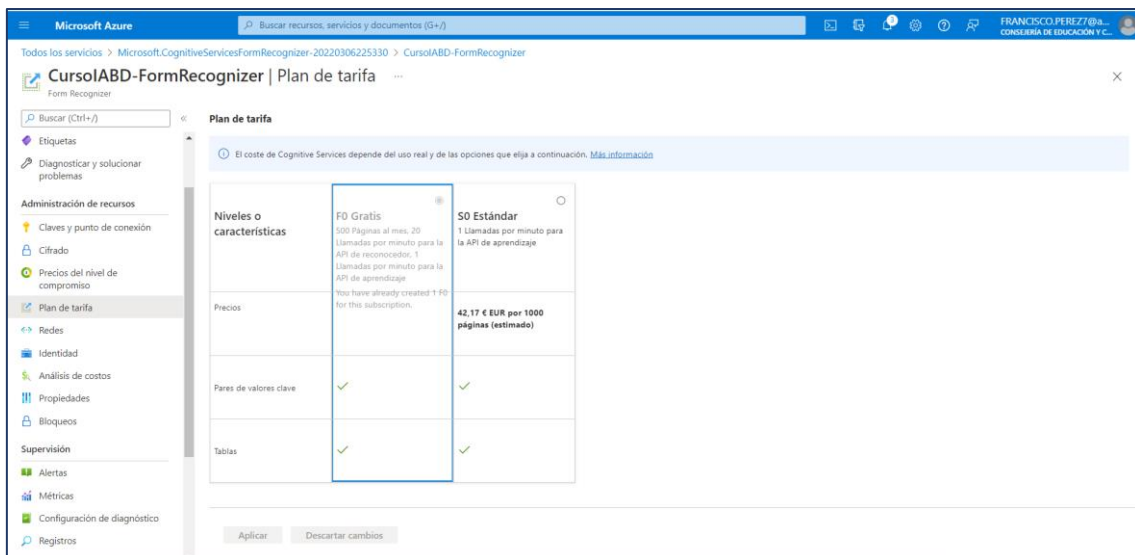
{}

En la pantalla de inicio del servicio creado, tenemos resumida toda la información del mismo. Desde aquí puede consultarse el **endpoint** asignado al servicio y las claves de acceso:



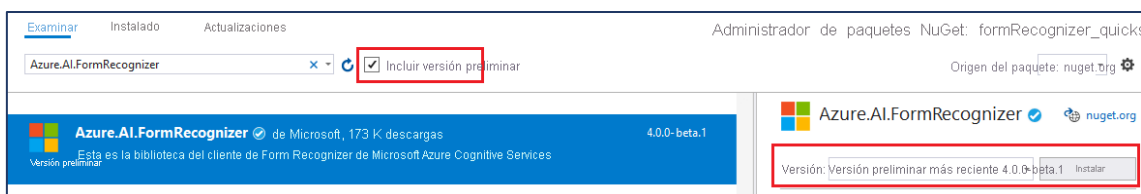
También puede obtenerse esta información desde el apartado "Claves y punto de conexión":





Una vez creado el servicio y su recurso asociado, para integrarlo en un programa, es necesario referenciar el paquete NuGet del API de reconocimiento: **Azure.AI.FormRecognizer**. En este documento utilizaremos la versión 4.0.0-beta.3

Al tratarse de un paquete en versión preliminar, hemos de seleccionar la opción correspondiente en el gestor de paquetes NuGet:



Para interactuar con el servicio, es necesario disponer del *endpoint* (que en la interfaz web aparece traducido al español como "Extremo" o "Punto de conexión", dependiendo de la pantalla) y las claves de API que se generan al activar el servicio.

Con independencia del modelo o de la funcionalidad de la que queramos hacer uso, hay que comenzar creando una instancia de la clase **DocumentAnalysisClient**. Para poder crear esta instancia es necesario instanciar los siguientes objetos:

- Un objeto de la clase **AzureKeyCredential** con la clave (clave 1) que se ha generado al activar el servicio en el portal web de Azure.
- También necesitamos la URI correspondiente al endpoint del servicio que hemos creado en Azure.

El código generado hasta este punto quedaría así:

```
C#  
using Azure;  
using Azure.AI.FormRecognizer.DocumentAnalysis;  
  
namespace FormRecognizer  
{  
    public class Program  
    {  
        public static async Task Main(string[] args)  
        {  
            const string Endpoint =  
                "https://cursoiabd-formrecognizer.cognitiveservices.azure.com/";  
  
            const string ApiKey = "a84158d10281481d98697c0d8a41b1eez";  
  
            AzureKeyCredential credential = new AzureKeyCredential(ApiKey);  
            DocumentAnalysisClient client = new DocumentAnalysisClient  
            (  
                new Uri(Endpoint),  
                credential  
            );  
        }  
    }  
}
```

6.1. USO DEL MODELO "GENERAL DOCUMENT"

El modelo de documento general es el más versátil y permite extraer texto, tablas, estructura, pares clave-valor y entidades con nombre de un documento.

El servicio toma como entrada una URI que representa el documento a analizar. Para este ejemplo, tomaremos el documento de prueba que se encuentra en esta ubicación: <https://raw.githubusercontent.com/Azure-Samples/cognitive-services-REST-api-samples/master/curl/form-recognizer/sample-layout.pdf>

Una implementación típica sería la siguiente:



```
// [... código del apartado anterior ...]

// URI del documento que se analizará
Uri fileUri = new Uri
(
    "https://raw.githubusercontent.com/Azure-Samples/cognitive-services-REST-api-samples/master/curl/form-recognizer/sample-layout.pdf"
);

// Inicia la operación de análisis
AnalyzeDocumentOperation operation = await client.StartAnalyzeDocumentFromUriAsync
(
    // Este es el id del modelo. Para modelos preentrenados, pueden consultarse los
    // ID de los distintos modelos en https://aka.ms/azsdk/formrecognizer/models, o
    // bien en https://docs.microsoft.com/en-us/azure/applied-ai-services/form-recognizer/quickstarts/try-v3-rest-api#analyze-document .

    // Esto huele a Python y huele mal. En un futuro es de esperar que este parámetro
    // deje de ser un string y se convierta en una enumeración (como Dios manda).
    "prebuilt-document",
    fileUri
);

// ¡¡¿?!! Nuevamente, vuelve a oler mal... Tiene pinta de que la biblioteca que
// opera por debajo no soporta programación asíncrona (TPL) y han usado un modelo
// Asynchronous Programming Model (APM, desaconsejado, por cierto).

// result contiene los resultados del análisis.
AnalyzeResult result = await operation.WaitForCompletionAsync();

Console.WriteLine("Detected key-value pairs:");

// Recorre el conjunto de pares clave-valor encontrados en el documento.
foreach (DocumentKeyValuePair kvp in result.KeyValuePairs)
{
    if (kvp.Value == null)
    {
        Console.WriteLine($" Found key with no value: '{kvp.Key.Content}'");
    }
    else
    {
        Console.WriteLine
            ($" Found key-value pair: '{kvp.Key.Content}' and '{kvp.Value.Content}'");
    }
}

// [... continúa ...]
```



```
Console.WriteLine("Detected entities:");

// Recorre el conjunto de entidades (campos predefinidos en el modelo, por ser muy
// habituales en muchos documentos) que se han encontrado
foreach (DocumentEntity entity in result.Entities)
{
    if (entity.SubCategory == null)
    {
        Console.WriteLine
            ("Found entity '{entity.Content}' with category '{entity.Category}'.");
    }
    else
    {
        Console.WriteLine
            (
                $"Found entity '{entity.Content}' with category '{entity.Category}' and sub-
                category '{entity.SubCategory}'."
            );
    }
}

// Recorre la estructura del documento y muestra los 'bounding boxes'
// de los elementos y marcas de selección (campos sí/no) reconocidos
foreach (DocumentPage page in result.Pages)
{
    Console.WriteLine
    (
        $"Document Page {page.PageNumber} has {page.Lines.Count} line(s), {page.Words.Count}
        word(s),"
    );
    Console.WriteLine($"and {page.SelectionMarks.Count} selection mark(s).");

    for (int i = 0; i < page.Lines.Count; i++)
    {
        DocumentLine line = page.Lines[i];
        Console.WriteLine($"Line {i} has content: '{line.Content}'");

        Console.WriteLine($"Its bounding box is:");
        Console.WriteLine
            ("Upper left => X: {line.BoundingBox[0].X}, Y= {line.BoundingBox[0].Y}");
        Console.WriteLine
            ("Upper right => X: {line.BoundingBox[1].X}, Y= {line.BoundingBox[1].Y}");
        Console.WriteLine
            ("Lower right => X: {line.BoundingBox[2].X}, Y= {line.BoundingBox[2].Y}");
        Console.WriteLine
            ("Lower left => X: {line.BoundingBox[3].X}, Y= {line.BoundingBox[3].Y}");
    }

    // [... continúa ...]
```



```
for (int i = 0; i < page.SelectionMarks.Count; i++)
{
    DocumentSelectionMark selectionMark = page.SelectionMarks[i];

    Console.WriteLine($" Selection Mark {i} is {selectionMark.State}.");
    Console.WriteLine($" Its bounding box is:");
    Console.WriteLine
    (
        $" Upper left => X: {selectionMark.BoundingBox[0].X}, Y=
        {selectionMark.BoundingBox[0].Y}"
    );
    Console.WriteLine
    (
        $" Upper right => X: {selectionMark.BoundingBox[1].X}, Y=
        {selectionMark.BoundingBox[1].Y}"
    );
    Console.WriteLine
    (
        $" Lower right => X: {selectionMark.BoundingBox[2].X}, Y=
        {selectionMark.BoundingBox[2].Y}"
    );
    Console.WriteLine
    (
        $" Lower left => X: {selectionMark.BoundingBox[3].X}, Y=
        {selectionMark.BoundingBox[3].Y}"
    );
}

// Muestra los resultados de análisis de texto manuscrito del documento
foreach (DocumentStyle style in result.Styles)
{
    bool isHandwritten = style.IsHandwritten.HasValue && style.IsHandwritten == true;

    if (isHandwritten && style.Confidence > 0.8)
    {
        Console.WriteLine($"Handwritten content found:");

        foreach (DocumentSpan span in style.Spans)
        {
            Console.WriteLine
            ($" Content: {result.Content.Substring(span.Offset, span.Length)}");
        }
    }
}

// Recorre las tablas encontradas en la estructura del documento.
Console.WriteLine("The following tables were extracted:");

for (int i = 0; i < result.Tables.Count; i++)
{
    DocumentTable table = result.Tables[i];
    Console.WriteLine
    ($" Table {i} has {table.RowCount} rows and {table.ColumnCount} columns.");

    foreach (DocumentTableCell cell in table.Cells)
    {
        Console.WriteLine
        ($" Cell ({cell.RowIndex}, {cell.ColumnIndex}) has kind '{cell.Kind}' and
        content: '{cell.Content}'");
    }
}
```


Aunque en la documentación del método `StartAnalyzeDocumentFromUriAsync` de la clase `DocumentAnalysisClient` se especifica la URL en la que obtener la cadena correspondiente al parámetro ***prebuilt model ID***, en dicha ubicación no se encuentra documentado (esto se habría evitado utilizando una enumeración, en vez de un string mágico, que es una repugnante costumbre en Python).

A partir de la documentación recogida en <https://docs.microsoft.com/en-us/azure/applied-ai-services/form-recognizer/quickstarts/try-v3-rest-api#analyze-document> pueden extraerse los id disponibles para cada modelo:

Modelo	Prebuilt model ID
General document	prebuilt-document
Layout	prebuilt-layout
Invoice	prebuilt-invoice
Receipt	prebuilt-receipt
ID Document	prebuilt-idDocument
W-2 tax document	prebuilt-tax.us.w2
Custom	<identificador designado en la creación del modelo>

6.2. OTROS MODELOS

En los siguientes enlaces puedes encontrar ejemplos de implementación de los modelos de Layout y de identificación de documentos a partir de modelos preentrenados:

- <https://docs.microsoft.com/en-us/azure/applied-ai-services/form-recognizer/quickstarts/try-v3-csharp-sdk#layout-model>
- <https://docs.microsoft.com/en-us/azure/applied-ai-services/form-recognizer/quickstarts/try-v3-csharp-sdk#prebuilt-model>



- Hemos de identificar el nombre de esta columna puesto que será necesario referirnos a ella más adelante.



```
using Microsoft.ML;

namespace TestingLR
{
    class TrainingProgram
    {
        static void Main (string[] args)
        {
            // 1. Preparación del contexto ML
            MLContext mlContext = new MLContext(seed: 0);
        }
    }
}
```



- Ten en cuenta que las columnas se numeran comenzando desde cero, en el mismo orden en que aparecen en el dataset.